

Your badge is live. How does anyone actually find it?

A browser on the other side of the world has to find your one server among billions of machines. Today is about addresses – IP addresses, subnets, CIDR, and DNS – the postal system of the internet. No building today: sit back, this one is all theory.

IN PARTNERSHIP WITH



From one machine's address to a name anyone can type

IDEA 1 IP addresses & subnets

Every machine has a unique number. Subnets group nearby machines into neighborhoods.

IPv4 subnet

IDEA 2 CIDR notation

A slash-number that names a whole block of addresses at once.

/24 /16

IDEA 3 RFC1918

Three address ranges reserved for private networks – never on the public internet.

10.0.0.0/8
192.168.0.0/16

IDEA 4 DNS

Names instead of numbers. How your badge got a hostname for free.

dig sslip.io

All four ideas build on each other in order – keep this strip in mind as we go.

An IP address is a street address for a machine

IP address IPV4

A unique number identifying a machine on a network. The common form is four numbers, each 0-255.

Example: **47.253.1.100** – that could be yesterday's server.

Subnet NEIGHBORHOOD

A group of addresses that can talk to each other directly, without leaving the neighborhood. Your server's vSwitch from Day 1 is a subnet.

Network vs host

Every address splits into two parts: the network part (which neighborhood) and the host part (which house in it).

Everything else today – CIDR, RFC1918, even DNS – is built on top of this one split: network part, then host part.

CIDR: one slash-number names a whole block

CIDR notation describes a block of addresses with a slash number, like 192.168.0.0/24. The number after the / says how many bits are fixed as the network part. The rest are free for hosts. An IPv4 address is 32 bits total.

CIDR	FIXED BITS	FREE BITS	ADDRESSES
/32	32	0	1 (a single host)
/24	24	8	256
/16	16	16	65,536
/8	8	24	16,777,216
/0	0	32	the entire internet

Rule of thumb
Free host bits = 32 – /number. Addresses = 2^(free bits). Bigger /number = **smaller**, more specific block.

Reading a block
192.168.0.0/24 means "addresses 192.168.0.0 through 192.168.0.255" – the first three numbers locked, the last free.

Bigger /number, smaller block – feel it, don't just memorize it



RFC1918: three ranges reserved for private networks

10.0.0.0/8	(10.x.x.x)
172.16.0.0/12	(172.16.x.x – 172.31.x.x)
192.168.0.0/16	(192.168.x.x)

Exactly three ranges

Memorize them – you will see them everywhere. Your VPC's CIDR block comes from one of these.

Private ≠ reachable

Your server has a private IP from one of these ranges (used inside the VPC) and the public EIP you attached on Day 1 (used from the internet). That is why the EIP was needed – the private address alone can't be reached from outside.

DNS is the internet's phone book

DNS TRANSLATOR

Nobody remembers **47.253.1.100**.
DNS (Domain Name System) translates
human names into IP addresses.

badge.example.com –DNS lookup→
47.253.1.100

sslip.io THE TRICK YOU ALREADY USED

<ip-with-dashes>.sslip.io
answers any lookup with the IP baked
into the name itself. That is how your
badge got a real hostname – and
HTTPS – with zero DNS setup on Day 2.

Watch a lookup happen

Run this from any terminal – no server
needed:

nslookup github.com
dig +short github.com

The DNS record on your badge's certificate is Caddy – from Day 2 – proving to the world it owns that name before it hands you a padlock.

Fast recall – work these from the rule of thumb, no tools

1. How many addresses are in 10.0.0.0/16 ? → _____
2. How many addresses are in 172.20.5.0/24 ? → _____
3. Is 192.168.1.50 inside the block 192.168.1.0/24 ? (yes/no) → _____
4. Is 10.5.0.1 a private (RFC1918) address? (yes/no) → _____
5. Which is a bigger block: /20 or /24 ? → _____

Hint: addresses in a block = $2^{(32 - \text{/number})}$. For "is this IP inside?", check whether the fixed network part matches.

You can read a CIDR block and explain where your server's names come from

Addresses & subnets

Every machine has a unique IP; subnets group nearby ones into neighborhoods.

CIDR, cold

Given any `/number`, you can compute the block size and read its range without a calculator.

After the break

You'll build the network pieces themselves – a VPC, a subnet, and the security-group rule you opened on faith on Day 2.

 Kahoot: Day 3 · Session 1 – IP addresses, CIDR math, RFC1918 ranges, and DNS.

The security-group rule you opened on faith.

Now you'll understand it.

Still a think day – no lab, nothing to build. You will understand exactly how your VPC, your subnet, and the firewall around your server fit together, and finally know what those two rules from Day 2 actually did.

VPC, then vSwitch – the same nesting as CIDR

VPC VIRTUAL PRIVATE CLOUD

Your own private, walled-off network inside Alibaba's cloud. You pick its address range – a CIDR block, like `192.168.0.0/16`. Nothing outside gets in unless you allow it.

vSwitch SUBNET

A subnet inside the VPC – a smaller CIDR slice that lives in one Availability Zone. Your server plugs into a vSwitch.

```
VPC 192.168.0.0/16    (your whole
private network)
└─ vSwitch 192.168.1.0/24  (a
   subnet in Zone A)
      └─ your ECS instance lives
         here
```

This is the exact nesting from the CIDR slider last session – a bigger block containing a smaller, more specific one.

A security group is a list of rules: port, protocol, source

A security group is a firewall wrapped around your instance – a list of rules saying which traffic is allowed in (inbound) and out (outbound). By default, almost everything inbound is blocked, which is why a brand-new server is unreachable until you open something.

This is the thing you opened on Day 2. The inbound rules for **80** and **443** were exactly this.

RULE PART	EXAMPLE	MEANING
Port	443	Which "door" – see ports, next slide
Protocol	TCP	The kind of traffic
Source	0.0.0.0/0	Who's allowed – 0.0.0.0/0 = anyone (that /0 from last session!)

Same port, different rule – and everything changes

THE INTERNET
0.0.0.0/0
knocking on :22

SECURITY

:22

:80

:443

GROUP

YOUR SERVER

SSH • :22

HTTP • :80

HTTPS • :443

rules

22 • 0.0.0.0/0

80 • 0.0.0.0/0

443 • no rule

from

the internet

inside the VPC

knock on

:22 SSH

:80 HTTP

:443 HTTPS

▶ Send a request

Turki's starting rules: :22 and :80 open to **anyone**, :443 has **no rule**. Press **Send a request** to :443 and watch exactly where it dies.

Ports let one machine run many services at once

PORT	SERVICE
22	SSH (how you log in)
80	HTTP (plain web)
443	HTTPS (secure web – your badge!)

Remember `docker run -p 80:80` from Day 2? That was port forwarding – a port on the host connected to a port inside the container. Traffic arriving at the server's door 443 gets forwarded to Caddy inside the container. The security-group rule is what let it arrive at all.

Three ideas you should recognize, not configure this week

Load balancer

Sits in front of many servers and spreads incoming traffic across them, so no single one is overwhelmed.

Proxy

A middle-man that forwards requests. Caddy is acting as one – forwarding web requests to your files.

CDN CONTENT DELIVERY NETWORK

Copies of your content cached in cities worldwide, so users load it from somewhere nearby and fast.

💡 On Alibaba these map to SLB (load balancer) and CDN – good to know the names, not something you'll set up this week.

He can SSH in fine. Nobody can load the site. Why?

Turki's current inbound rules:

Port	Protocol	Source
22	TCP	0.0.0.0/0
80	TCP	0.0.0.0/0

← SSH works

1. Which rule is missing so the browser can load the HTTPS site?

Port: _____ Protocol: _____ Source: _____

2. In one sentence, why can Turki still SSH in even though the site won't load?

Hint: he can SSH (port 22) but not load the site (port 443). Which door is missing from the rules? What must the **source** be for "anyone on the internet"?

5 tables, 5 cloud engineers each – no two players, same game

1 • Huddle

Each table talks it over and divides up the 9 games from the lineup – one game per player.

2 • No repeats at your table

All 5 picks at a table must be unique.
Two tables can pick the same game;
two players at the same table cannot.

3 • Deploy

Each engineer pulls and runs their own choice on their own VM, on the port listed for that game.

🤝 Teammates can help each other – during the challenge, and after finishing your own deploy, go help someone at your table who hasn't. No blaming: this is a fun learning exercise.

15-minute clock. Winner: the table with the most VMs live in a browser when time's up – first table to 5 / 5 wins outright.

Press start once every table has picked

15 : 00

TIME REMAINING – FIRST TABLE TO 5 / 5 WINS OUTRIGHT

▶ Start

⏸ Pause

↺ Reset (15:00)

Nine games, one port each – find yours below

GAME	IMAGE	PORT
Tetris	bsord/tetris:latest	7603
Super Mario	pengbai/docker-supermario:latest	4911
Pac-Man	nbaillie/pacman:latest	5962
Cats-as-a-Service	alialjaffer/caas:latest	6767
2048	aashiqq/2048-game:latest	4857
Snake	aschil/snake:latest	5813
Clumsy Bird	alialjaffer/clumsy-bird:latest	9042
Donkey Kong	magicalyak/docker-games:donkeykong	3319
Lion King	magicalyak/docker-games:lionking	8264

Hint: docker pull <your game's image> – nothing else. Same rules as Day 2 and this morning.

You can design a VPC, read a security-group rule, and explain why it matters

This morning

IP addresses, subnets, CIDR math, RFC1918 ranges, and how DNS turns a name into an address.

This afternoon

VPCs, vSwitches, security-group rules, ports, and the concepts behind load balancing and CDNs.

Tomorrow

Instead of running everything yourself, you'll let the cloud manage services for you – starting with storage (OSS), then scaling your app on Kubernetes.

 Kahoot: Day 3 · Session 2 – VPCs, vSwitches, security groups, ports, and load balancing/CDN concepts.

IN PARTNERSHIP WITH

